

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3– Hackathon

Course Code:	DSA0404	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	C05-Hackathon	Assessment:	Assessment Tool 3–Hackathon
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Deva Dharshini	Reg. No.:	192472350
Section / Batch:	BE-CSE-AI,2024	Date:	31-08-2026

Code of Conduct

I Deva Dharshini(192472350) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Deva Dharshini

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Problem Statement:

Sales Forecasting for Retail. Predict monthly/quarterly sales for a retail store using historical sales, promotions, and seasonality as features via multiple linear regression.

Source Code:

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# PAGE SETTINGS
# =====

st.set_page_config(
    page_title="Retail Sales Forecasting",
    page_icon="📊",
    layout="wide"
)

# =====
# TITLE
# =====

st.title("📊 Retail Sales Forecasting System")

st.write(
    "Predict monthly retail sales using historical sales, "
    "promotions and seasonality with Multiple Linear Regression."
)
```

```

# =====
# LOAD DATA
# =====

@st.cache_data
def load_data():

    data = pd.read_csv("sales_data.csv")

    data["Date"] = pd.to_datetime(data["Date"])

    data = data.sort_values("Date")

    return data

try:

    df = load_data()

except FileNotFoundError:

    st.error(
        "sales_data.csv was not found. "
        "Please place sales_data.csv in the same folder as app.py."
    )

    st.stop()

# =====
# DATA PREPROCESSING
# =====

```

```
df = df.drop_duplicates()
```

```
df = df.dropna(  
    subset=["Date", "Sales", "Promotion"]  
)
```

```
# =====  
# FEATURE ENGINEERING  
# =====
```

```
df["Month"] = df["Date"].dt.month
```

```
df["Quarter"] = df["Date"].dt.quarter
```

```
df["Year"] = df["Date"].dt.year
```

```
# Previous month's sales
```

```
df["Previous_Sales"] = df["Sales"].shift(1)
```

```
# Three-month moving average
```

```
df["Sales_MA_3"] = (  
    df["Sales"]  
    .rolling(window=3)  
    .mean()  
)
```

```
# Remove rows containing NaN values
```

```
df = df.dropna().reset_index(drop=True)
```

```
# =====  
# SIDEBAR  
# =====
```

```
st.sidebar.header("🔑 Forecast Settings")
```

```
forecast_promotion = st.sidebar.slider(
    "Promotion (%)",
    min_value=0,
    max_value=100,
    value=20
)
```

```
forecast_month = st.sidebar.selectbox(
    "Forecast Month",
    list(range(1, 13)),
    index=0
)
```

```
forecast_quarter = (
    (forecast_month - 1) // 3
) + 1
```

```
# =====
# FEATURES AND TARGET
# =====
```

```
features = [
    "Previous_Sales",
    "Promotion",
    "Month",
    "Quarter",
    "Sales_MA_3"
]
```

```
X = df[features]
```

```
y = df["Sales"]
```

```
# =====
```

```
# TRAIN / TEST SPLIT
```

```
# =====
```

```
split_index = int(len(df) * 0.8)
```

```
X_train = X.iloc[:split_index]
```

```
X_test = X.iloc[split_index:]
```

```
y_train = y.iloc[:split_index]
```

```
y_test = y.iloc[split_index:]
```

```
# =====
```

```
# TRAIN MODEL
```

```
# =====
```

```
model = LinearRegression()
```

```
model.fit(
```

```
    X_train,
```

```
    y_train
```

```
)
```

```
# =====
```

```
# TEST PREDICTION
```

```
# =====
```

```
y_pred = model.predict(
```

```

        X_test
    )

# =====
# MODEL EVALUATION
# =====

mae = mean_absolute_error(
    y_test,
    y_pred
)

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

r2 = r2_score(
    y_test,
    y_pred
)

# =====
# DASHBOARD METRICS
# =====

col1, col2, col3, col4 = st.columns(4)

with col1:

```

```
st.metric(
    "Average Sales",
    f"₹{df['Sales'].mean():.0f}"
)
```

with col2:

```
st.metric(
    "MAE",
    f"₹{mae:.0f}"
)
```

with col3:

```
st.metric(
    "RMSE",
    f"₹{rmse:.0f}"
)
```

with col4:

```
st.metric(
    "R² Score",
    f"{r2:.2f}"
)
```

```
# =====
# HISTORICAL SALES TREND
# =====
```

```
st.subheader("📊 Historical Sales Trend")
```



```
fig1 = px.line(  
    df,  
    x="Date",  
    y="Sales",  
    title="Monthly Sales"  
)
```

```
st.plotly_chart(  
    fig1,  
    use_container_width=True  
)
```

```
# =====  
# PROMOTION ANALYSIS  
# =====
```

```
st.subheader("🔗 Promotion vs Sales")
```

```
fig2 = px.scatter(  
    df,  
    x="Promotion",  
    y="Sales",  
    trendline="ols",  
    title="Impact of Promotion on Sales"  
)
```

```
st.plotly_chart(  
    fig2,  
    use_container_width=True  
)
```

```
# =====
```

```
# MONTHLY SEASONALITY
```

```
# =====
```

```
st.subheader("📅 Monthly Seasonality")
```

```
monthly_sales = (  
    df.groupby("Month")["Sales"]  
    .mean()  
    .reset_index()  
)
```

```
fig3 = px.bar(  
    monthly_sales,  
    x="Month",  
    y="Sales",  
    title="Average Sales by Month"  
)
```

```
st.plotly_chart(  
    fig3,  
    use_container_width=True  
)
```

```
# =====
```

```
# ACTUAL VS PREDICTED
```

```
# =====
```

```
st.subheader("📊 Actual vs Predicted Sales")
```

```
comparison = pd.DataFrame({  
  
    "Date": df.iloc[split_index:]["Date"].values,  
  
    "Actual Sales": y_test.values,
```

```

        "Predicted Sales": y_pred

    })

fig4 = px.line(
    comparison,
    x="Date",
    y=["Actual Sales", "Predicted Sales"],
    title="Actual vs Predicted Sales"
)

st.plotly_chart(
    fig4,
    use_container_width=True
)

# =====
# FUTURE SALES FORECAST
# =====

st.subheader("🔮 Future Sales Forecast")

latest_previous_sales = (
    df["Sales"].iloc[-1]
)

latest_moving_average = (
    df["Sales_MA_3"].iloc[-1]
)

future_data = pd.DataFrame({

```

```

    "Previous_Sales": [
        latest_previous_sales
    ],

    "Promotion": [
        forecast_promotion
    ],

    "Month": [
        forecast_month
    ],

    "Quarter": [
        forecast_quarter
    ],

    "Sales_MA_3": [
        latest_moving_average
    ]

})

```

```

future_prediction = model.predict(
    future_data
)[0]

```

```

st.success(
    f" 🎉 Predicted Sales: ₹{future_prediction:,.2f}"
)

```

```

# =====

```

```
# WHAT-IF PROMOTION ANALYSIS
```

```
# =====
```

```
st.subheader("💡 What-If Promotion Analysis")
```

```
promotion_values = [
```

```
    0,
```

```
    10,
```

```
    20,
```

```
    30,
```

```
    40,
```

```
    50
```

```
]
```

```
what_if_data = pd.DataFrame({
```

```
    "Previous_Sales":
```

```
        [latest_previous_sales] *
```

```
        len(promotion_values),
```

```
    "Promotion":
```

```
        promotion_values,
```

```
    "Month":
```

```
        [forecast_month] *
```

```
        len(promotion_values),
```

```
    "Quarter":
```

```
        [forecast_quarter] *
```

```
        len(promotion_values),
```

```
    "Sales_MA_3":
```

```
        [latest_moving_average] *
```

```
        len(promotion_values)

    })

what_if_predictions = model.predict(
    what_if_data
)

what_if_result = pd.DataFrame({

    "Promotion (%)":
        promotion_values,

    "Predicted Sales":
        what_if_predictions

})

fig5 = px.bar(
    what_if_result,
    x="Promotion (%)",
    y="Predicted Sales",
    title="Predicted Sales for Different Promotion Levels"
)

st.plotly_chart(
    fig5,
    use_container_width=True
)
```

```
# =====  
# BUSINESS RECOMMENDATION  
# =====
```

```
st.subheader("💡 Business Recommendation")
```

```
best_index = (  
    what_if_result["Predicted Sales"]  
    .idxmax()  
)
```

```
best_promotion = (  
    what_if_result.loc[  
        best_index,  
        "Promotion (%)"  
    ]  
)
```

```
best_sales = (  
    what_if_result.loc[  
        best_index,  
        "Predicted Sales"  
    ]  
)
```

```
st.info(  
    f"Based on the model, a "  
    f"{best_promotion}% promotion gives the highest "  
    f"predicted sales of ₹{best_sales:,.2f}."  
)
```

```
# =====  
# DATASET  
# =====
```

```
with st.expander("📄 View Dataset"):
```

```
    st.dataframe(  
        df,  
        use_container_width=True  
    )
```

```
# =====  
# MODEL INFORMATION  
# =====
```

```
with st.expander("📖 About the Model"):
```

```
    st.write(  
        """  
        Algorithm: Multiple Linear Regression  
  
        Target Variable:  
        Sales  
  
        Features:  
        • Previous Sales  
        • Promotion  
        • Month  
        • Quarter  
        • Three-Month Moving Average  
  
        Evaluation Metrics:  
        • MAE
```


- RMSE
- R^2 Score

The data is split chronologically so that historical data is used for training and later data is used for testing.

"""

)

Output:

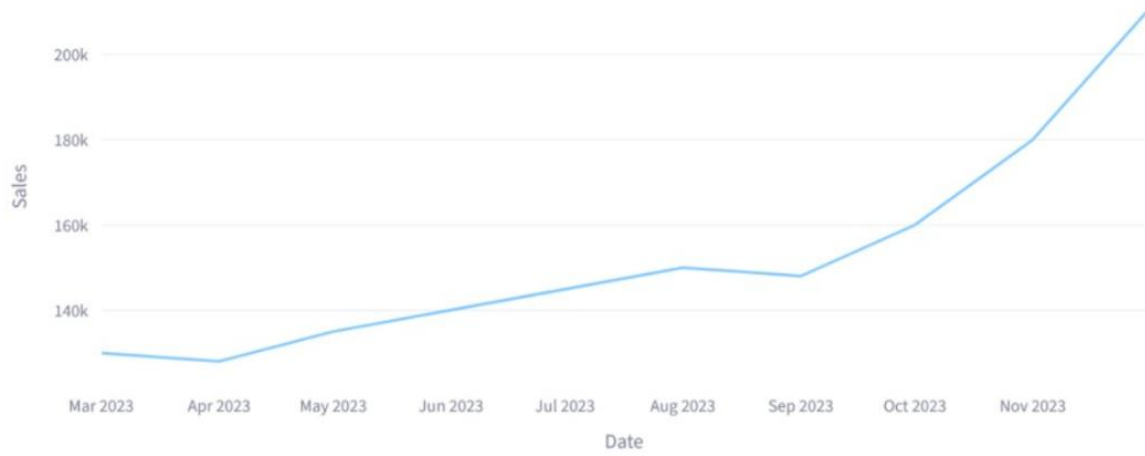




Historical Sales Trend

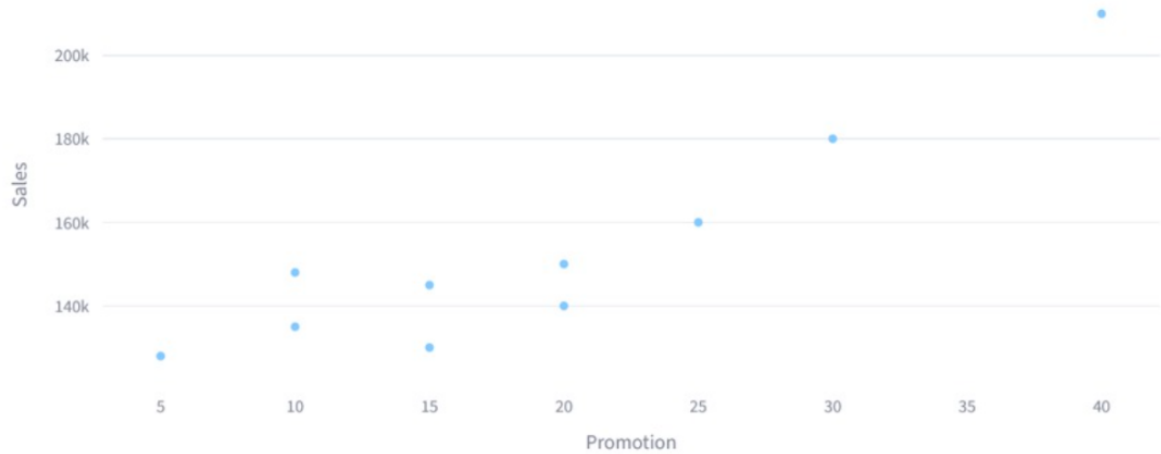


Monthly Sales



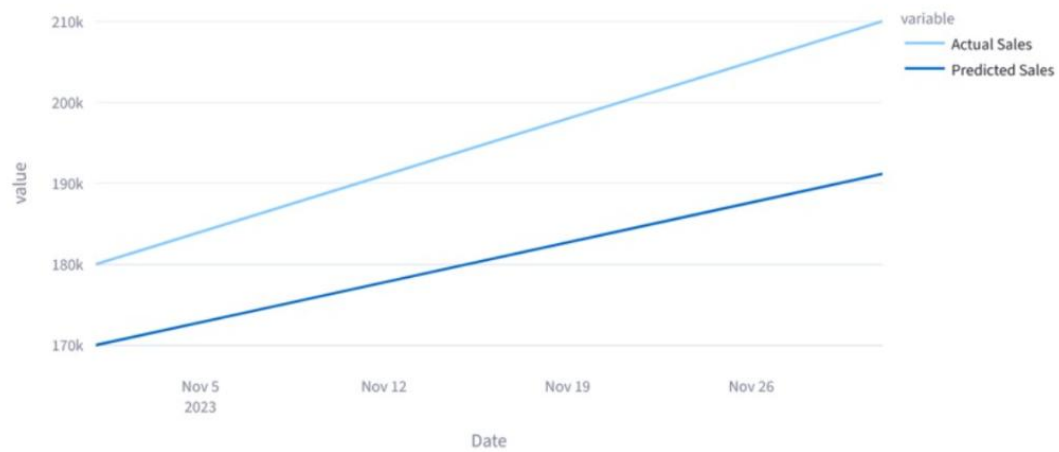
Promotion vs Sales

Impact of Promotion on Sales



Actual vs Predicted Sales

Actual vs Predicted Sales



Geo Tag Photo:



Mevalurkuppam, Tamil Nadu, India

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026482° Long 80.017009°
Monday, 31/08/2026 10:02 AM GMT +05:30